Parallel Connected Components
High-Performance Computing 2025/2026

Salvatore Gilles Cassarà, Alberto Cimmino
GitHub Repo: https://github.com/Iron16Bit/ParallelConnectedComponents

1 Introduction

1.1 Problem Statement

In graph theory, a connected component of an undirected graph G is a connected subgraph of G.

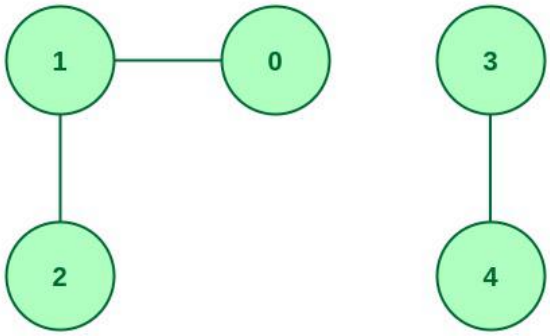

Figure 1: Example of an undirected graph with 2 connected components. Source: GeeksForGeeks

The connected components problem is used in many real-time applications like image segmentation, clustering of 3D points or for tracking communities in social media streams. Based on the liveliness of these applications, the problem needs to be quickly solved. This report presents a sequential implementation for the problem and uses parallelization to speed it up.

1.2 Sequential Implementation

A preliminary analysis of the sequential implementations for the connected components problem showed a solution based on *Union-Find Algorithms* to be the fastest [4]. In particular, Rem's algorithm manages to outperform other Union-Find algorithms even though it is an old algorithm [2, 5].

In order to prepare for the parallel implementation, we also implemented SV's algorithm, as many parallel algorithms like FastSV [7] are based in this algorithm.

Since the parallel design is based on SV's algorithm (Algorithm 1), this section will focus on its description. The algorithm is split into three phases:

1. **Stochastic Hooking**: this phase consists in checking all neighbors of a node and finding the minimum grandparent among the neighbor's grandparents;
2. **Shortcutting**: the tree is flattened by setting the parent to the minimum between parent and grandparent;
3. **Convergence Check**: check if the old and new grandparents are different. If they are the same, the algorithm has converged and we can stop.

This process allowed us to hook nodes to the minimun grandparent of the tree they are rooted in (their connected component), labeling them in a sense. We now need to count the unique labels in order to compute the number of connected components, which is done as described in Algorithm 2.

Algorithm 1 SV Connected Components Algorithm

Require: $graph = (V, E)$, $parent \in \mathbb{R}^{\mathbb{N}}$

```
1: procedure ConnectedComponentsSV(graph, parent)
2:     parentNext ← InitParent(graph)
3:     stop ← false
4:     while not stop do
5:         stop ← true
                                                    ▷ Tree hooking (stochastic)
6:         for u = 0 to graph.numberOfNodes − 1 do
7:             for k = 0 to graph.degree[u] − 1 do
8:                 v ← graph.neighbors[u][k]
9:                 if parentNext[parent[u]] < parent[parent[v]] then
10:                    parentNext[parent[u]] ← parent[parent[v]]
11:                end if
12:            end for
13:        end for
                                                    ▷ Shortcutting
14:        for u = 0 to graph.numberOfNodes − 1 do
15:            if parentNext[u] < parent[parent[u]] then
16:                parentNext[u] ← parent[parent[u]]
17:            end if
18:        end for
                                                    ▷ Convergence check and update
19:        for i = 0 to graph.numberOfNodes − 1 do
20:            if parent[parent[i]] ≠ parentNext[parentNext[i]] then
21:                stop ← false
22:            end if
23:            parent[i] ← parentNext[i]
24:        end for
25:    end while
26: end procedure
```

Algorithm 2 Print Solution

Require: $parent \in \mathbb{R}^{\mathbb{N}}$, $length \in \mathbb{N}$

```
1: procedure PrintSolution(parent, length)
2:     uniqueParents ← new array of size length
3:     numberOfUniqueParents ← 0
4:     for i = 0 to length − 1 do
5:         vertexRoot ← Root(i, parent)
6:         if numberOfUniqueParents = 0 or not Contains(0, numberOfUniqueParents, vertexRoot,
   uniqueParents) then
7:             uniqueParents[numberOfUniqueParents] ← vertexRoot
8:             numberOfUniqueParents ← numberOfUniqueParents + 1
9:         end if
10:    end for
11:    print "Number of connected components: " numberOfUniqueParents
12: end procedure
```

2 Parallel Design

2.1 State of the Art

The last few years have seen several improvements to the Connected Components problem through the creation of new algorithms and approaches. One of the first is the addition of an *adaptive* component to SV's Algorithm [4]. This approach consists in behaving differently based on the graph's diameter: if it is small, a BFS is more efficient than SV's algorithm and vice versa. Heuristics are used to choose which approach to use based on the graph topology. Further improvements come from FastSV [6, 7], a parallel implementation of SV's algorithm which simplifies the original algorithm, adds new hooking strategies and maps all operations to linear algebra operations which are executed through specific libraries like GraphBLAS[1] or CombBLAS[2]. Finally, other solutions map

[1] https://graphblas.org/

[2] https://github.com/PASSIONLab/CombBLAS

the solution to the *Contour* algorithm [3], which achieves even better performances than FastSV.

2.2 Parallelization Strategies

From the examination of Algorithm 1, we can notice that there are two main points that would benefit from parallelization:

- The entire `ConnectedComponentsSV` function: it is possible to split the graph into subgraphs in order to make each MPI process call the function on its subgraph;
- The various loops over all the nodes in the graph / subgraph, which can be parallelized over various threads using openMP, as they can all be performed independently.

On the other hand, parallelization of this code is met with a big problem: for the algorithm to advance, all processes need to know the whole *parentNext* vector which is used for computing convergence and as the *parent* vector in the next cycles. This is translated into a big communication among all MPI processes to make sure that every MPI process has the updated *parentNext* vector.

In the sequential version, the graph is stored as a 2D matrix. For parallelization it is possible to switch to a **CSR-like** storage format, which stores the (sparse) adjacency matrix through three array:

- **row offset**: an array of length equals to the number of rows + 1. Row offset[i] gives the index in the other arrays where row i starts; row offset[i+1] - row offset[i] is the number of non-zeros in row i. Called *degree* in the Graph struct;
- **column indices**: an array listing the column index of each non-zero element, stored row by row. Called *neighbors* in the Graph struct;
- **values**: an array storing the corresponding non-zero values, aligned one-to-one with column indices. This one is not necessary as the value will necessary be 1 if the entry is present in the adjacency matrix.

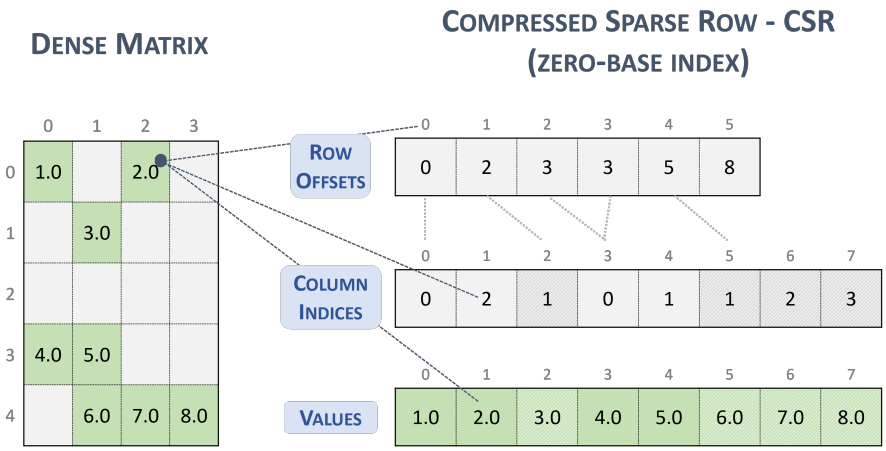

Figure 2: Example of CSR storage format. Source: NVIDIA

Having the graph stored in a flat array structure allows easier communication as `MPI_Scatterv` can be used to split it across the various processes and this also guarantees contiguous memory layout, reducing message overhead.

3 Implementation

The first step for parallelizing a problem is understanding how to split the data structure, in this case the graph, among the different processes. We used the split function described in Algorithm 3, which finds the ideal number of edges each process should handle based on the total number of edges and the total number of processes and assigns rows of the adjacency matrix to processes trying to stay as close as possible to that ideal number. Rows of the adjacency matrix aren't split

Algorithm 3 Graph Split by Edge Balance

Require: *graph*, *size* **Ensure:** *sendCountsNode*, *sendCountsDegree*

```
1: procedure SPLIT(graph, size, sendCountsNode, sendCountsDegree)
2:     ideal ← ⌈graph.numberOfEdges/size⌉
3:     sentNodes ← 0
4:     assignedEdges ← 0
5:     for p = 0 to size − 2 do
6:         localEdges ← 0
7:         localNodes ← 0
8:         while sentNodes < graph.numberOfNodes do
9:             candidate ← localEdges + graph.degree[sentNodes]
10:            if localEdges = 0 or |candidate − ideal| < |localEdges − ideal| then
11:                localEdges ← candidate
12:                sentNodes ← sentNodes + 1
13:                localNodes ← localNodes + 1
14:            else
15:                break
16:            end if
17:        end while
18:        sendCountsNode[p] ← localNodes
19:        sendCountsDegree[p] ← localEdges
20:        assignedEdges ← assignedEdges + localEdges
21:    end for
                                  ▷ Assign remaining nodes and edges to last process
22:    sendCountsNode[size − 1] ← graph.numberOfNodes − sentNodes
23:    sendCountsDegree[size − 1] ← graph.numberOfEdges − assignedEdges
24: end procedure
```

because this allows processes to have complete and unique access to nodes, knowing all of their neighbors and not risking overwrites in the vector of parents of those nodes from other processes.

Once the data is divided among all processes by the process with rank 0 (which will be called the *master* process), the master process sends a `MPI_Bcast` message to all nodes with the information about how many data each process will receive and then sends that data through several `MPI_Scatter` and `MPI_Scatterv` (based on whether each node will receive the same amount of data or different amount of data). The received data is finally used by each MPI process to build a *SubGraph* as described in Algorithm 4.

Algorithm 4 SubGraph Data Structure

```
1: structure SUBGRAPH
2:     numberOfNodes          ▷ Number of vertices in the subgraph
3:     numberOfEdges          ▷ Number of edges in the subgraph
4:     degree[]               ▷ Degree of each node
5:     neighbors[][]          ▷ Adjacency lists
6:     offset                 ▷ Global node index offset
```

Algorithm 5 describes our implementation of FastSV, a parallel and faster version of SV's algorithm described in Algorithm 1. The non-parallel improvements of FastSV compared to SV are:

- **Neighbor-wise propagation**: computes, for each vertex, the minimum current grandparent label among itself and its neighbors. This allows to more quickly converge to the minimum parent which will be used to label the connected component;
- **Aggressive Hooking**: a cheap element-wise operation which reuses results from the stochastic hooking to hook a node onto another tree when they belong to the same connected components in order to shorten the path to the root (node with lowest ID);
- **Pointer-Jumping**: the grandparent array is updated each iteration to reduce tree height and make both propagation and hooking faster.

The described parallel code takes great advantage of OpenMP parallelization because it is dominated by loops whose iterations are mostly independent (the main exception is stochastic hooking, which can create conflicting updates and therefore requires synchronization as discussed

Algorithm 5 Hybrid Batched Connected Components

Require: SubGraph $graph$, parent vector f, $totalNodes$

procedure ConnectedComponents($graph$, f, $totalNodes$)
  $n \leftarrow graph.numberOfNodes$
  Allocate vectors gp, dup, $mngp$, $buffer$ of size $totalNodes$
  ▷ Parallel initialization
  OpenMP parallel for
  for $i = 0$ **to** $totalNodes - 1$ **do**
    $gp[i] \leftarrow f[i]$
    $dup[i] \leftarrow gp[i]$
    $mngp[i] \leftarrow gp[i]$
    $buffer[i] \leftarrow f[i]$
  end for
  $global\_sum \leftarrow 1$
  ▷
  OpenMP parallel region
  while $global\_sum \neq 0$ **do**
    for $step = 1$ **to** $BATCH\_SIZE$ **do**
      ▷ 1. Neighbor-wise propagation: $mngp \leftarrow graph \otimes gp$
      `findMinGrandparentOfNeighbours(graph, gp, mngp)`
      ▷ 2. Stochastic hooking
      OpenMP parallel for
      $f[f[u]] \leftarrow$ `atomic(` $\min(f[f[u]], mngp[u])$`)`
      ▷ 3. Aggressive hooking + shortcutting
      OpenMP parallel for
      $f[u] \leftarrow \min(f[u], mngp[u], gp[u])$
      ▷ 4. Global parent synchronization
      OpenMP master
      if $step = BATCH\_SIZE$ **then**
        MPI_Allgatherv
        $f \leftarrow buffer$
      end if
      OpenMP barrier
      ▷ 5. Pointer-Jumping
      OpenMP parallel for
      $gp[u] \leftarrow \min(f[f[u]], buffer[f[u]])$
      ▷ 6. Global convergence check
      OpenMP master
      if $step = BATCH\_SIZE$ **then**
        $local\_diff \leftarrow (gp \neq dup)$
        MPI_Allreduce($local\_diff, global\_sum, +$)
      else
        $global\_sum \leftarrow 1$
      end if
      ▷ 7. Update state
      OpenMP parallel for
      $dup[u] \leftarrow gp[u]$
    end for
  end while
  Free auxiliary vectors
end procedure

in Section 3.1). On the other hand, MPI communication constitutes the main scalability bottleneck: during grandparent computation and propagation we perform random-access global indexing (e.g., evaluating $f[f[u]]$), so each process needs a periodically consistent global view of the parent vector f, including entries owned by other processes.

For this reason, every K local iterations we synchronize f using `MPI_Allgatherv`. Each process contributes only its owned slice $f[offset : offset + n]$, and `Allgatherv` reconstructs the full vector on all ranks. This pattern matches our data distribution (disjoint contiguous slices) and reduces communication compared to collective operations in which every rank must send $O(totalNodes)$ elements. In contrast, `MPI_Allreduce` with a `MIN` operator would implement an element-wise reduction and would be appropriate only if multiple processes could update the same $f[i]$ and we wanted to combine overlapping contributions; this is not the case here, since each entry of f is

owned and updated by exactly one rank between synchronizations.

Finally, we still use `MPI_Allreduce` for convergence detection only: each rank computes a local "changed" flag and we sum these flags across processes to decide termination. To further reduce communication cost, the global `Allgatherv` synchronization is performed only once every $K =$ `BATCH_SIZE` iterations (we use $K = 4$ empirically). Larger K values may delay synchronization too much; moreover, because termination is checked only at batch boundaries, an excessively large K can cause the last batch to perform unnecessary extra work.

3.1 Data Dependencies

Table 1 describes the variables used in the `ConnectedComponents()` function with respect to the OpenMP parallelization, describing their types of access and visibility.

Variable	Type of Access	Visibility
$f[\,]$	Read / Write	Shared
$gp[\,]$	Read / Write	Shared
$dup[\,]$	Read / Write	Shared
$mngp[\,]$	Read / Write	Shared
$buffer[\,]$	Read / Write	Shared
global_sum	Read / Write	Shared
local_diff	Read / Write	Private (only master)
reduce_result	Read / Write	Private (only master)

Table 1: Variables access patterns and visibility

We can quickly notice that variables *buffer*[], *global_sum*, *local_diff* and *reduce_result* are not problematic as the first two are only written by the master thread and read by the others and the other two are private to the various threads. The accesses to *gp*[], *dup*[] and *mngp*[] all happen sequentially in the various loop without any dependency, allowing to easily parallelize the loop without any problem.

The first problematic case is the access to $f[]$ during stochastic hooking, as it is impossible to predict the accessed position of each iteration, and such positions could be the same, causing write-write overlaps among different threads. Based on the assumption that such overlaps will not happen frequently, this operation is done atomically with `__sync_bool_compare_and_swap()`, which checks the value written in *f[pos]* and if it is still that value when trying to write, it overwrites it as no other thread is writing at the same time, otherwise retries by fetching the new *f[pos]* value.

The second problematic case is again the access to $f[]$ between global parent synchronization and pointer-jumping. If the master thread hasn't completed overwriting the $f[]$ array after the communication when the other threads begin performing pointer-jumping there can be read-write conflicts. This problem has been solved with an **OpenMP barrier** among the two steps to guarantee that writes on $f[]$ are over when pointer-jumping is performed.

4 Performance and Scalability Analysis

In order the evaluate the performances of the parallel implementation we chose a *standard* graph size and evaluated it on the *standard*, *doubled* and *halved* graph size as described in more details in Table 2. It is important to note that these graphs have been generated randomly by an in-house algorithm, with no particular patterns or properties other than having a fixed number of nodes split into a chosen amount of connected components.

Table 2: Graph datasets used in experiments

Name	# Nodes	# Edges	# CCs	Size
double	100,000,000	396,975,220	629,462	6.57 GB
standard	50,000,000	178,548,916	4,750,265	2.92 GB
halved	25,000,000	92,019,612	1,750,265	1.47 GB

The algorithm has been tested on UniTrento's *hpc2* cluster, with an "exclusive pack" placing strategy, as only one process needs to perform I/O operations and having more process on the same machine allows a better inter-process communication. Each execution was repeated 10 times to get averaged results and the code has been tested only using MPI, only using OpenMP and

with MPI + OpenMP. In order to properly test the algorithm, we decided to run it with 2, 4, 8, 16, 32 and 64 workers each time, as that allowed us to see how the performances changed while increasing the number of processes. We decided to stop at 64, as that is where the performance started to worsen. The evaluation is based on these three following metrics:

- **Execution Time (T)**: total wall-clock time required to complete the computation.
- **Speedup (S)**: ratio between sequential and parallel execution time,

$$S = \frac{T_{\text{seq}}}{T_{\text{par}}}.$$

- **Efficiency (E)**: speedup normalized by the number of threads,

$$E = \frac{S}{p} = \frac{T_{\text{seq}}}{T_{\text{par}} \cdot p},$$

  where p is the number of threads.

Moreover, the algorithms have been tested in the three following situations: MPI parallelization, OpenMP parallelization and Hybrid OpenMP+MPI parallelization, divided for execution times (described in Figure 3), speedup (described in Figure 4) and efficiency (described in Figure 4). It is important to notice that the execution of the program can be divided into three steps:

1. **Setup**: I/O time for reading the file, create the graph and send it to the various processes;
2. **Computation**: compute the parent array through the FastSV algorithm;
3. **Solution Reconstruction**: iterate through the parent array and count the number of unique labels to obtain the number of connected components.

The Setup phase takes similar times in both sequential and parallel executions and its bottleneck is the I/O time needed to read the file describing the graph from memory. No speedup is gained from the parallel version of the algorithm in this case, but improvements can be obtained by upgrading the cluster's memory. At the same time, the Solution Reconstruction phase is exactly the same both for the sequential and the parallel version and takes exactly the same time. For these reasons, the following analysis only focuses on the Computation phase, which is a comparison between the sequential SV algorithm 1 and the parallel FastSV algorithm 5.

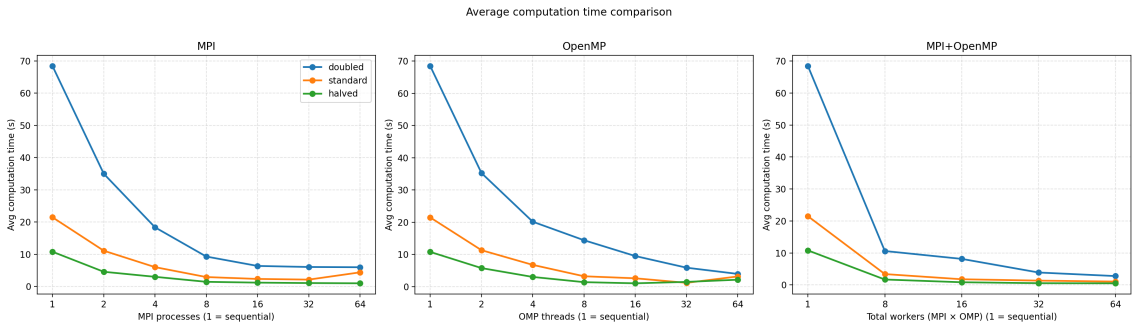

Figure 3: Execution Time of the parallel algorithm with: MPI-only, OpenMP-only and Hybrid parallelization

We can start our analysis from the execution times described in Figure 3. All parallel executions are faster than a sequential execution, but just as expected they all follow *Amdahl's law*: "the overall performance improvement gained by optimizing a single part of a system is limited by the fraction of time that the improved part is actually used" [1], meaning that with the increase of threads/processes the improvements to the execution times tend to be reduced (deminishing returns), or in same cases they even get worse like when using 64 processes or threads and only parallelizing with MPI or OpenMP. The next thing to notice is that both the MPI-only and the OpenMP-only parallelization reduce the execution times more slowly compared to the hybrid OpenMP+MPI parallelization, whose curve has a much steeper slope. Such difference will be described later, but it already shows that the hybrid parallelization might be favoured to the other two.

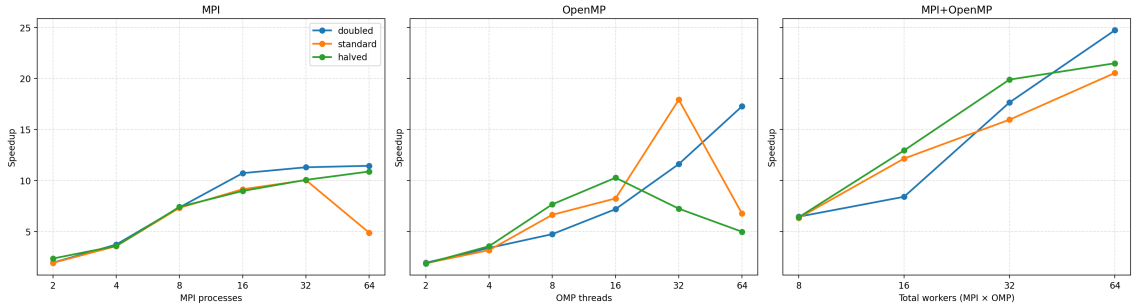

Figure 4: Speedup of the parallel algorithm with: MPI-only, OpenMP-only and Hybrid parallelization

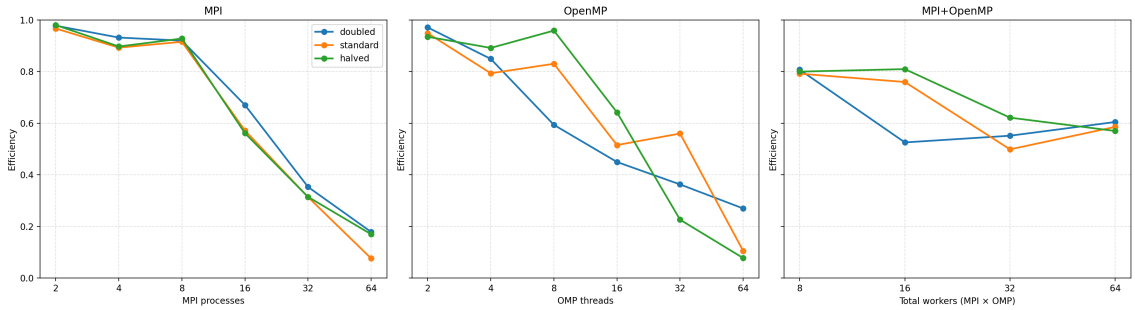

Figure 5: Efficiency of the parallel algorithm with: MPI-only, OpenMP-only and Hybrid parallelization

The next criteria to analyse is the speedup described in Figure 4, which is how much faster the parallel algorithm is compared to the sequential one. Just like what we have seen from Figure 3, with the increase of processes or threads the speedups hit a plateau or even start decreasing. Again, this is obvious for the MPI-only and OpenMP-only parallelization where the speedups tank with too many processes or threads while they are still increasing for the hybrid parallelization.

Finally, we can analyze the efficiency from Figure 5. Both the executions with only OpenMP threads and in particular the one with only MPI processes are decreasing with a really steep slope: the efficiency is kind of stable for a while with the increase of processes or threads up until a point, then it tanks. This happens because in both cases work isn't distributed evenly and efficiently: in the OpenMP-only parallelization the various threads do not work on a subgraph but on the whole graph, meaning that there is an extremely high contention for the atomic access to the $f[]$ array; MPI-only parallelization struggles even more because it suffers from the main disadvantage discussed in Section 3 of the global `Allgatherv` without the benefits of speeding up the various steps thanks to the OpenMP parallelization. As expected, the hybrid OpenMP+MPI parallelization is instead the most efficient one as it greately balances the advantages and disadvantages of both implementations: it reduces the OpenMP threads contention by dividing the graph into various subgraphs for the MPI processes, it doesn't create too many MPI processes and the execution time of the various MPI processes is reduced by the OpenMP parallelization. This results in the lowest efficiency reaching 0.5 rather than 0.1.

	Halved	**Standard**	**Doubled**
8	0.799662	0.791772	0.807045
16	0.809456	0.759525	0.525578
32	0.621516	0.498760	0.551511
64	0.569914	0.585663	0.604609

Table 3: Summary of the efficiency of the Hybrid parallel algorithm on each graph and with each configuration

Table 3 summarizes the efficiency of the parallel implementation over the various graphs and configurations. Unexpectedly, we can notice that efficiency increases with the decrease of the graph size. With High-Performance Computing and parallelization performances usually increase with the increase of size as more data can be split into more processes and / or threads, granting higher

efficiency. Unfortunately it is not the case for this algorithm due to the `Allgetherv` that acts as a bottleneck: the more data there is to communicate, the more time it takes, having a negative effect on performances.

Table 3 can also be used to discuss about scalability. Basing the analysis on the following definitions:

- **Weak Scalability**: The ability of a program to maintain constant efficiency when the problem size increases at the same rate as the number of process;
- **Strong Scalability**: The ability of a program to maintain constant efficiency as the problem size is kept constant.

When considering the *Doubled* size graph, we have a big jump when going from 8 to 16 workers but then efficiency stabilizes, resulting in almost a strong scalability. Something similar can be said about the *Standard* and *Halved* graphs where the 8 and 16 configurations have similar efficiency and so do the 32 and 64 configurations, but there is a gap among these two couples of configurations. Regarding weak scalability, considering the diagonal (*Halved*, 8), (*Standard*, 16) and (*Doubled*, 32); we can see that the first two entries of the diagonal have extremely similar efficiencies, but there is a gap between the second and the third, leading us to believe that our application is not weakly scalable.

5 Conclusions

The proposed parallelization of the connected components problem through the FastSV algorithm achieves an average efficiency of 66%. The general criteria for deciding if a parallelization is good or bad is whether it achieves at least an efficiency of 70%. Even though the algorithm we presented isn't exactly above that threshold, we think that it a valid solution considering two main facts: as described in Section 2.1, many modern algorithms use extremely efficient graph libraries like GraphBLAS or CombBLAS, which also provide parallelization, but we avoided using any library and instead implemented our own functions; the connected components problem is inherently difficult to parallelize because it is a "global problem", as each parallel unit needs to know the information about the whole graph, forcing synchronization. Such communication and synchronization are much more than the needed computation, making this problem communication/synchronization-heavy rather than computation-heavy.

Taking all these information into consideration, we believe we presented a valid parallel solution to the connected components problem which allows to speedup computations up to 25 times on matrices with 100 millions of elements. With future improvements on the communication like different communication patterns that can avoid a global `Allgatherv` for synchronization or data compression, the proposed algorithm can achieve a much higher efficiency and be considered a successful parallel algorithm for the connected components problem.

References

[1] Gene M. Amdahl. Validity of the single processor approach to achieving large scale computing capabilities. In *Proceedings of the April 18-20, 1967, Spring Joint Computer Conference*, AFIPS '67 (Spring), page 483–485, New York, NY, USA, 1967. Association for Computing Machinery.

[2] Edsger Wybe Dijkstra. *A discipline of programming*, volume 613924118. prentice-hall Englewood Cliffs, 1976.

[3] Zhihui Du, Oliver Alvarado Rodriguez, Fuhuan Li, Mohammad Dindoost, and David A. Bader. Contour algorithm for connectivity. In *2023 IEEE 30th International Conference on High Performance Computing, Data, and Analytics (HiPC)*, pages 66–75, 2023.

[4] Chirag Jain, Patrick Flick, Tony Pan, Oded Green, and Srinivas Aluru. An adaptive parallel algorithm for computing connected components. *IEEE Transactions on Parallel and Distributed Systems*, 28(9):2428–2439, 2017.

[5] Md Mostofa Ali Patwary, Jean Blair, and Fredrik Manne. Experiments on union-find algorithms for the disjoint-set data structure. In *International Symposium on Experimental Algorithms*, pages 411–423. Springer, 2010.

[6] Yongzhe Zhang, Ariful Azad, and Aydın Buluç. Parallel algorithms for finding connected components using linear algebra. *Journal of Parallel and Distributed Computing*, 144:14–27, 2020.

[7] Yongzhe Zhang, Ariful Azad, and Zhenjiang Hu. *FastSV: A Distributed-Memory Connected Component Algorithm with Fast Convergence*, pages 46–57.